

Computational Logic

Sandra Alves - DCC/FCUP

2026/27

Logistics of the course

- Teaching team:
 - Lectures: Sandra Alves
 - Lab classes: Sandra Alves and Nelma Moreira
- Webpage on Moodle
<https://moodle2627.up.pt/course/view.php?id=6631>
(slides, exercises, etc...)
- Evaluation
 - First test (FT) + Second test (ST)

$$Final = FT * (0.5) + ST * (0.5)$$

where PT , $ST \geq 6$ and $Final \geq 9.5$.

- Resit exam on the contents of the whole course.
- First test: November 4 (to be confirmed)
- Second test: during the exam period

Some bibliography

- *“Logic in Computer Science”*, Michael Huth , 1962.
- *“Language, proof, and logic”*, Jon Barwise, 1999.
- *“Essentials of logic programming”*, Christopher John Hogger, 1990.
- *“Rigorous software development”*, José Bacelar Almeida et al., 2011.
- *“Mathematical Logic for Computer Science”*. Mordechai Ben-Ari 2001.

Objectives and Program

Objectives

Introduction to logic from a computational perspective.

- basic notions on logic reasoning and use of deductive systems.
- relation between semantic validity and deduction: soundness and completeness (what is valid can be proved; what is proved is valid)...
- propositional logic and first order (predicate) logic
- axiomatizations
- breve introduction to logic programming (prolog).

Program

1. Propositional logic

- syntax; semantics (tautology, satisfiability, validity); normal forms; the natural deduction system (DN); soundness and completeness of DN ; other deductive systems (Hilbert, tableaux, resolution); decidability and satisfiability algorithms

2. First order logic (predicate logic)

- languages; syntax; semantics (interpretation, models, satisfiability, validity); the natural deduction system for FOL (DN); soundness and completeness of DN ; theories and axiomatizations; undecidability of FOL; limitations of deductive systems (Gödel's incompleteness)

3. Logic Programming

- resolution; Horn formulas and programs; term unification, SLD-resolution.

What is logic?

Logic is the study of the methods and principles used to distinguish correct from incorrect reasoning.

Irving Copi

Logic is justly considered the basis of all other sciences, even if only for the reason that in every argument we employ concepts taken from the field of logic, and that every correct inference proceeds in accordance with its laws.

Alfred Tarskia

Logic is the art of thinking well; the mind, like the body, requires to be trained before it can use its powers in the most advantageous way.

Isacc Taylor



Logic is the beginning of wisdom, not the end.

Mr. Spock

A (very) brief history of logic

A long long time ago...

In ancient Greece

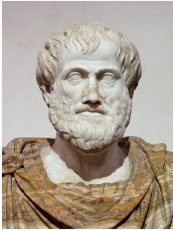


Aristotle (384-322 AC) is credited as the creator of formal logic, as a science that studies reasoning (valid or not):

- notion of proposition; conjunction, disjunction, implication and negation; quantifiers (the aristotelic forms); syllogisms

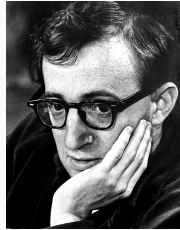
Example: *modus ponens* from $A \rightarrow B$ and A infer B

Some care is needed with syllogisms



All men are mortals
Socrates is a men

Therefore, Socrates is mortal



All men are mortals
Socrates was mortal

Therefore, all men are Socrates



What is mathematical logic?

Leibniz $\rightarrow$ Boole $\rightarrow$ Frege $\rightarrow$... $\rightarrow$ Hilbert $\rightarrow$ Gödel $\rightarrow$...



Symbolic formalisations associated with problems on the foundations of mathematics...

- model theory
- theory of deduction (or proof)
- axiomatic systems (e.g. set theory)

A variety of logics

- Classical logic: propositional and predicate logic (first order)
- Higher order logic
- Sub-structural (e.g. intuitionistic) logic: modification of the properties of logic connectives. Ex: $\neg\neg A \not\Rightarrow A$; $A \wedge A \not\Rightarrow A$
- Modal logic: need, knowledge, temporal, description, etc. The notion of truth is parameterized... Example: *Today is Monday*

What is “a logic”?

- a language of an alphabet in which it is defined what are **formulas** and other objects.
- A way to define semantics (=notion of what is true) for the formulas. The same logic can have several semantic interpretations...
- A deduction system that, using a finite number of rules allows us to obtain formulas from others (=reasoning). Must be sound: what it deduces must be true... But it will also be good that what is true must be deducible (complete)... Although this is not always possible.

Why is logic relevant to computer science?

Logic circuits: Boolean logic

Artificial intelligence: expert systems, natural language, web semantics, data mining, etc.

Finite Automata: second-order monadic logic with a successor.

XML: A structured document is a tree of a language represented by tree automata corresponding to formulas of second-order logic.

Algorithms and complexity: complexity classes of problems can be characterized by classes of logical formulas.

Programming Languages:

- logic programming (e.g. prolog)
- type theory (type systems=deductive systems)

Formal specification and verification:

- hardware verification (circuits)
- software correction tests (dynamic, temporal, etc.)
- cyber-systems
- security protocols
- multi-agent systems

Automated deduction: Specific theories (integers, arrays, lists, etc.); SAT/SMT solvers

Assisted Theorem Provers: Extracting programs from specifications (Isabelle, Coq, Agda, etc)

Propositional logic

Propositional logic

We consider declarative sentences (*propositions*), which can be *true* (**V**) or *false* (**F**)

- *Elephants are mammals* **V**
- *Lisbon is a city* **V**
- $2 + 3 = 7$ **F**
- $5 \in \mathbb{N}$ **V**
- $3 > 7$ **F**
- *A square has 6 sides* **F**

and we study the construction and the veracity of building of other propositions using connectives

or, and, no, if... then...

Propositional logic

- *Elephants are mammals and Lisbon is a city* **V**
because it is a **conjunction** of propositions **V**
- $2 + 3 = 7$ **or** $5 \in \mathbb{N}$ **V**
because it is a **disjunction** of propositions of which one is **V**
- **no** $3 > 7$ **V**
because it is a **negation** of a proposition **F**
- **If** $7 > 3$ **then** $3 + 3 = 6$ **V**
because a **implication** is **V** if the consequent is **V** whenever the antecedent is **V**
- **If** $3 > 7$ **then** $2 + 3 = 6$ **V**
because it is a **implication** whose antecedent is **F**

Logic Conectives

Each proposition will be represented by a *propositional variable* and the logical connectives by *n-ary symbols*:

Conective	Symbols	Arity	Other equivalent symbols
Conjunction	$\wedge$	2	$\&$, $\&\&$, $\cdot$
Disjunction	$\vee$	2	$ $, $+$
Negation	$\neg$	1	$\sim$, $\bar{}$, $!$
Implication	$\rightarrow$	2	$\Rightarrow$, $\supset$

Languages of propositional logic

A language of propositional logic is formed from the following sets of **primitive symbols** (alphabets):

- a countable set of propositional variables

$$\mathcal{V}_{Prop} = \{p, q, r, \dots, p_1, \dots\}$$

- logical connectives $\wedge$, $\vee$, $\neg$, $\rightarrow$
- the parentheses (and)

Formulas

A *well-formed formula* ($\phi, \psi, \theta, \dots$) is defined inductively by the following rules:

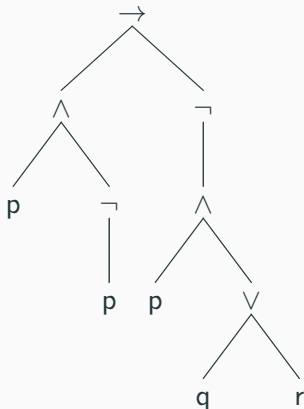
1. a propositional variable p is a formula
2. if ϕ is a formula then $(\neg\phi)$ is a formula
3. if ϕ and ψ are formulas then $(\phi \wedge \psi)$, $(\phi \vee \psi)$ and $(\phi \rightarrow \psi)$ are formulas

Example

$$((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$$

Syntactic tree of a formula (omitting parenthesis)

$$((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$$



Convention for omitting parentheses

- outer parentheses can be omitted
- $\neg$ takes precedence over $\wedge$
- $\wedge$ takes precedence over $\vee$
- $\vee$ takes precedence over $\rightarrow$
- $\wedge$ and $\vee$ are left associative
- $\rightarrow$ is right associative

Examples

$\phi \wedge \psi \vee \theta$ is an abbreviation of $((\phi \wedge \psi) \vee \theta)$

$\psi \wedge \phi \wedge \theta$ matches to $((\psi \wedge \phi) \wedge \theta)$

$p \wedge \neg p \rightarrow \neg(p \wedge (q \vee r))$ matches to $((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$

$p \rightarrow q \rightarrow r$ corresponds to $p \rightarrow (q \rightarrow r)$

Sub-formulas

An **immediate sub-formula** is defined inductively by:

1. a propositional variable does not have **immediate sub-formulas**
2. $(\neg\phi)$ has ϕ as **immediate sub-formula**
3. formulas $(\phi \wedge \psi)$, $(\phi \vee \psi)$ and $(\phi \rightarrow \psi)$ have ϕ and ψ as **immediate sub-formulas**

A formula ϕ is a **sub-formula** of a formula ψ if and only if:

1. ϕ is an immediate sub-formula of ψ
2. there is a formula θ such that ϕ is a sub-formula of θ and θ is a sub-formula of ψ

Which are the sub-formulas and immediate sub-formulas of:

$$((\neg p \wedge q) \rightarrow (p \wedge (q \vee \neg r)))$$

Semantics of propositional logic

Consider the **truth values** **V** and **F** .

Assignment of truth values

$v : \mathcal{V}_{Prop} \longrightarrow \{\mathbf{V}, \mathbf{F}\}$ which assigns a truth value to each propositional

An assignment v can be extended to the set of formulas:

1. for $p \in \mathcal{V}_{Prop}$ $v(p)$ is already set
2. not ϕ :

$$v(\neg\phi) = \mathbf{V} \quad \text{if} \quad v(\phi) = \mathbf{F}$$

$$v(\neg\phi) = \mathbf{F} \quad \text{if} \quad v(\phi) = \mathbf{V}$$

Semantics of propositional logic

3. ϕ and ψ :

$$\begin{array}{ll} v(\phi \wedge \psi) = \mathbf{V} & \text{if } v(\phi) = \mathbf{V} \text{ and } v(\psi) = \mathbf{V} \\ v(\phi \wedge \psi) = \mathbf{F} & \text{otherwise} \end{array}$$

4. ϕ or ψ :

$$\begin{array}{ll} v(\phi \vee \psi) = \mathbf{V} & \text{if } v(\phi) = \mathbf{V} \text{ or } v(\psi) = \mathbf{V} \\ v(\phi \vee \psi) = \mathbf{F} & \text{otherwise} \end{array}$$

5. if ϕ then ψ :

$$\begin{array}{ll} v(\phi \rightarrow \psi) = \mathbf{F} & \text{if } v(\phi) = \mathbf{V} \text{ and } v(\psi) = \mathbf{F} \\ v(\phi \rightarrow \psi) = \mathbf{V} & \text{otherwise} \end{array}$$

Truth tables

ϕ	$\neg \phi$
F	V
V	F

ϕ	ψ	$\phi \wedge \psi$
F	F	F
F	V	F
V	F	F
V	V	V

ϕ	ψ	$\phi \vee \psi$
F	F	F
F	V	V
V	F	V
V	V	V

ϕ	ψ	$\phi \rightarrow \psi$
F	F	V
F	V	V
V	F	F
V	V	V

If $v(p) = \mathbf{V}$, $v(q) = \mathbf{F}$ and $v(r) = \mathbf{V}$, we can calculate the value of $v((p \wedge q) \vee \neg r)$:

p	q	r	$(p \wedge q)$	$\vee$	$\neg r$
V	F	V	F	F	F

Truth table of a formula

A formula ϕ with n propositional variables has 2^n assignments. Why?

We can build a table in which each row corresponds to one of them:

p	q	r	(p $\wedge$ q)	$\vee$	$\neg$	r
V	V	V	V	V	F	
V	V	F	V	V	V	
V	F	V	F	F	F	
V	F	F	F	V	V	
F	V	V	F	F	F	
F	V	F	F	V	V	
F	F	V	F	F	F	
F	F	F	F	V	V	

Satisfiability, Tautologies and Contradictions

A formula ϕ is:

satisfiable if there is an assignment v such that $v(\phi) = \mathbf{V}$,
write $\models_v \phi$ and say that v satisfies ϕ

tautology if for all assignments v , $v(\phi) = \mathbf{V}$ and write $\models \phi$
Ex: $\models p \vee \neg p$ (Third excluded)

contradiction if for all assignment v , $v(\phi) = \mathbf{F}$ and write $\not\models \phi$.
Ex: $\not\models p \wedge \neg p$.

A formula is **unsatisfiable** if and only if it is a contradiction.